coursera

Hands-On-Learning: Enterprise Document System Refactoring: From Legacy Code to LangChain Architecture

Description

Apply systematic 5-step refactoring methodology (Audit, Map, Modularize, Test, Deploy) to transform an undocumented legacy document processing system into a maintainable LangChain application with comprehensive testing and zero-downtime deployment strategy.

Learning Objectives

By the end of this activity, learners will be able to:

- Conduct comprehensive code audits identifying hidden dependencies and technical debt patterns
- Create dependency maps that reveal coupling points and refactoring boundaries

- Apply incremental modularization strategies that minimize production risk
- Implement defensive testing with behavior verification achieving 80%+ code coverage

Assignment Components

Introductory Slide:

- **Title:** Lead Developer Emergency: Save DataAnalytics Corp from $2M Technical Debt
- **Subtitle:** Five days to refactor undocumented legacy system processing millions of financial documents

Tools to be used:

- LangChain (Python framework)
- pytest (testing framework)
- OpenAI API (LLM provider)
- GitHub Actions for CI/CD
- Python static analysis tools (pylint, mypy)

Scenario

You're the newly promoted Lead Developer at DataAnalytics Corp, a Fortune 500 financial services firm. The company's document classification system processes millions of regulatory documents daily but has become a critical business risk. Built 18 months ago by contractors who left no documentation, the 2,000-line codebase crashes weekly (costing $50,000/hour in lost productivity), takes 6 hours to make simple prompt changes due to tight coupling, and has zero test coverage. Last week's outage cost $2M in regulatory penalties. The CTO has

authorized complete refactoring—your mission: apply the proven 5-step methodology (Audit, Map, Modularize, Test, Deploy) to transform this system into a maintainable LangChain application. You must create a comprehensive refactoring blueprint, implement modular architecture with proper interfaces, achieve 80%+ test coverage, and design a zero-downtime deployment strategy. The executive team expects your plan by Friday.

Instructions for learners

Step 1: Audit Phase - Systematic Code Analysis

- Download the legacy document_processor.py codebase from provided repository
- Use auditing checklist: count prompts, identify API calls, document error handling gaps
- Run static analysis (pylint) to identify code quality issues and coupling patterns
- Document findings with line numbers, severity levels, and specific refactoring actions needed

Step 2: Map Phase - Dependency Visualization

- Create visual dependency map showing function call relationships
- Identify tightly coupled components and circular dependencies
- Mark untangle points where safe separation is possible
- Prioritize refactoring order based on risk assessment and business impact

Step 3: Modularize Phase - LangChain Implementation

- Extract prompts into separate PromptTemplate files following your dependency map
- Implement LangChain model abstractions with proper error handling and logging
- Create OutputParser classes with validation schemas using Pydantic
- Organize code into modular structure: prompts/, chains/, parsers/, utils/
- Document interfaces between components with clear contracts

Step 4: Test Phase - Comprehensive Testing Strategy

- Write unit tests for each LangChain component (prompts, parsers, chains)
- Create integration tests verifying component interaction and data flow
- Implement behavior verification tests comparing refactored vs original output
- Run pytest with coverage reporting, target 80%+ code coverage
- Document edge cases and failure scenarios with corresponding test cases

Step 5: Deploy Phase - Migration Strategy Documentation

- Design strangler fig deployment approach with gradual traffic shifting
- Create rollback plan for immediate reversion if issues arise
- Document monitoring metrics for production health validation
- Prepare deployment checklist with risk mitigation at each phase

Deliverable for Submission (AI Grading)

1 Deliverable 1: Comprehensive Refactoring Blueprint

Question Prompt: How did you apply the 5-step refactoring methodology (Audit, Map, Modularize, Test, Deploy) to transform the legacy document processor, and why does your systematic approach with dependency mapping and risk assessment minimize production disruption while achieving modularization goals?

Response Type: File Upload

File Types: Text (.txt, .pdf, .docx, .md), Image (.png, .jpg for dependency diagrams)

2 Deliverable 2: Modular LangChain Architecture

Question Prompt: What architectural decisions shaped your modular LangChain implementation (component organization, error handling, logging), and how does your design eliminate the tight coupling and brittle error handling that plagued the legacy system?

Response Type: File Upload

File Types: Coding files (.py, .json, .yml, .md), Compressed archives (.zip)

3 Deliverable 3: Test Coverage Suite

Question Prompt: How did you design your test strategy (unit tests, integration tests, behavior verification) to achieve 80%+ coverage and ensure the refactored code maintains identical behavior to the original system, and why does this testing approach protect against regression bugs during future modifications?

Response Type: File Upload

File Types: Coding files (.py for tests), Text (.txt, .html for coverage reports), Image (.png for coverage screenshots)

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.